

Introduction to AI

Friday 5 June 2026

Contributors: Huw Day, Pau Erola, James Thomas,
Richard Lane, Ben Westhenry

Course overview

- **What** is a neural network / **why** are they are useful
(theory)
- **How to build** a neural network in code
(coding)
- How neural networks **learn**
(theory)
- **Training** our neural network in code
(coding)

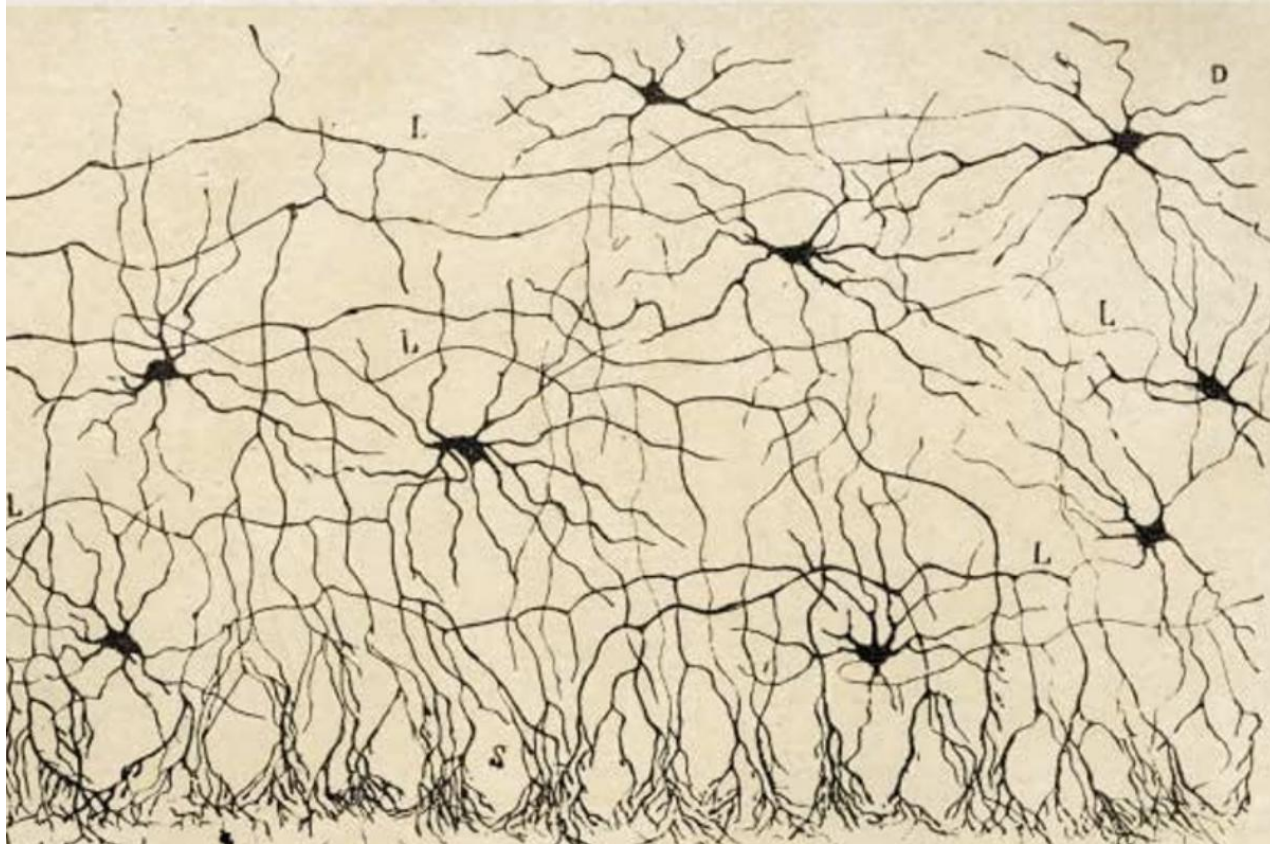
Demos and resources

All material for today is on the HackMD page:

<https://hackmd.io/@jgi-training>

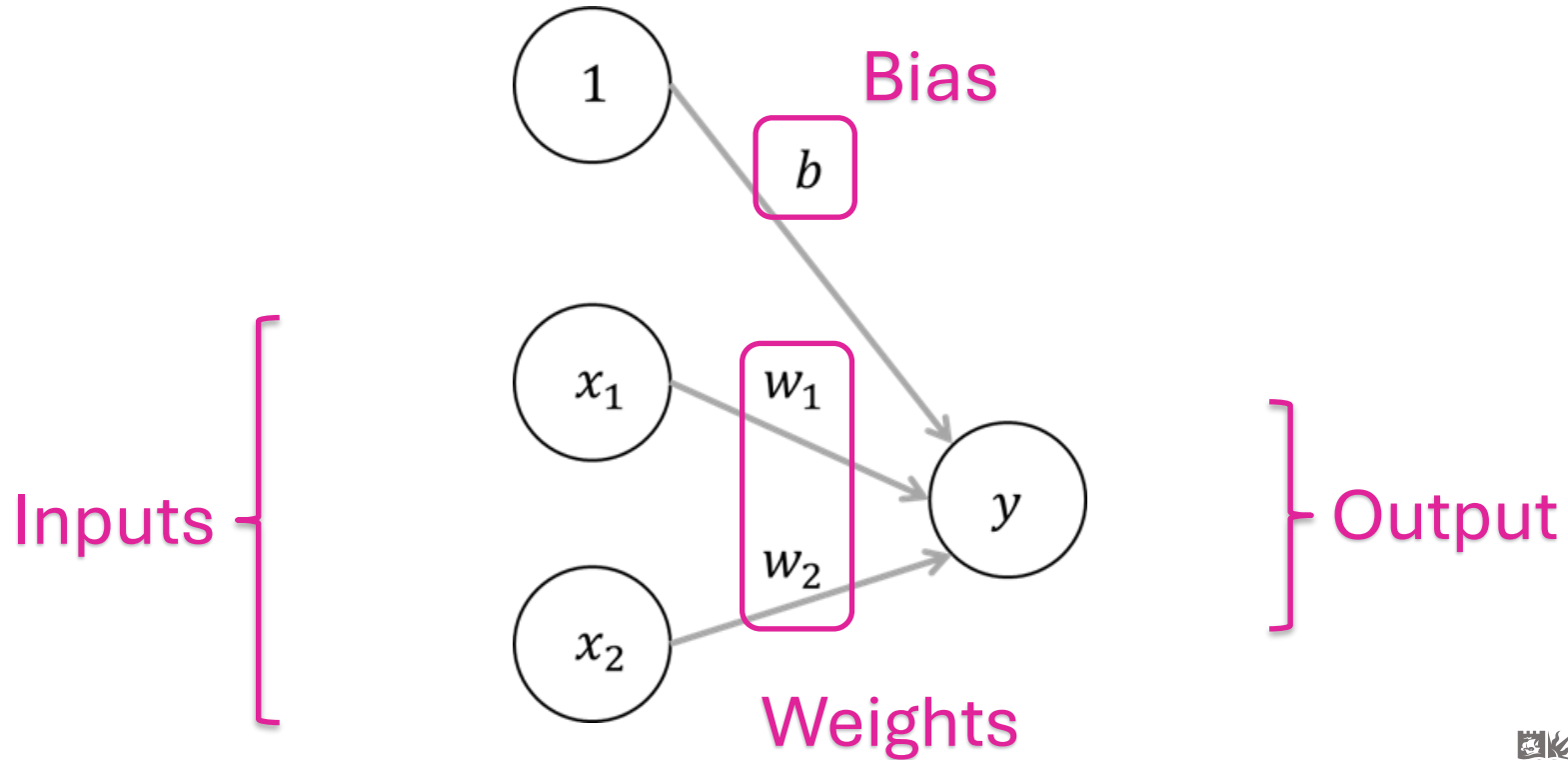
Including interactive demos, reference notes,
further reading

Inspiration for neural networks

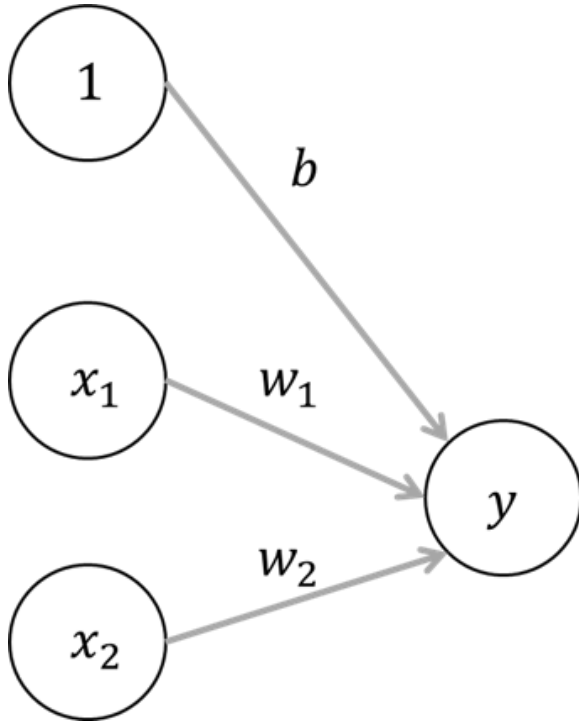


Fragment of Ramón y Cajal's first publication on the central nervous system, from Sotelo, C. *Viewing the brain through the master hand of Ramon y Cajal. Nat Rev Neurosci* 4, 71–77 (2003) <https://doi.org/10.1038/nrn1010>

The origin of neural networks The 'Perceptron'



The origin of neural networks The 'Perceptron'



$$y = w_1x_1 + w_2x_2 + b$$

or for classification...

$$y = \begin{cases} 0 & (w_1x_1 + w_2x_2 + b \leq 0) \\ 1 & (w_1x_1 + w_2x_2 + b > 0) \end{cases}$$

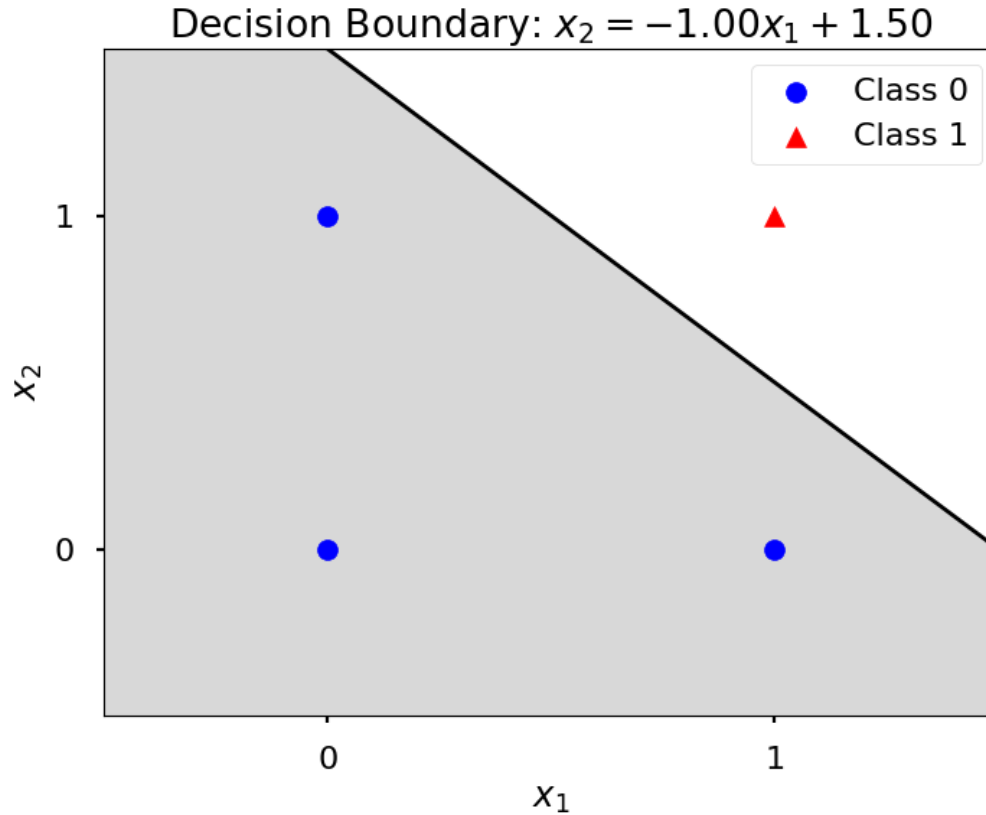
↑
Class label

Decision boundary

example: an AND gate



x_1	x_2	y (Class)
0	0	0
1	0	0
0	1	0
1	1	1

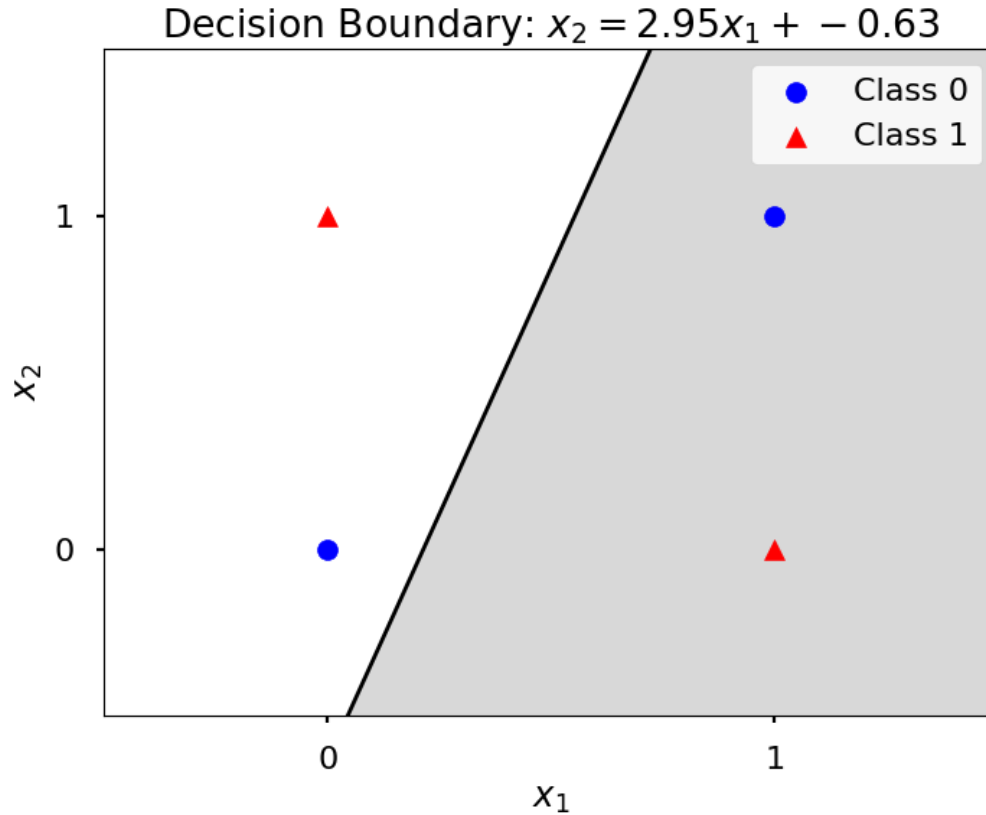


Decision boundary

example: an XOR gate

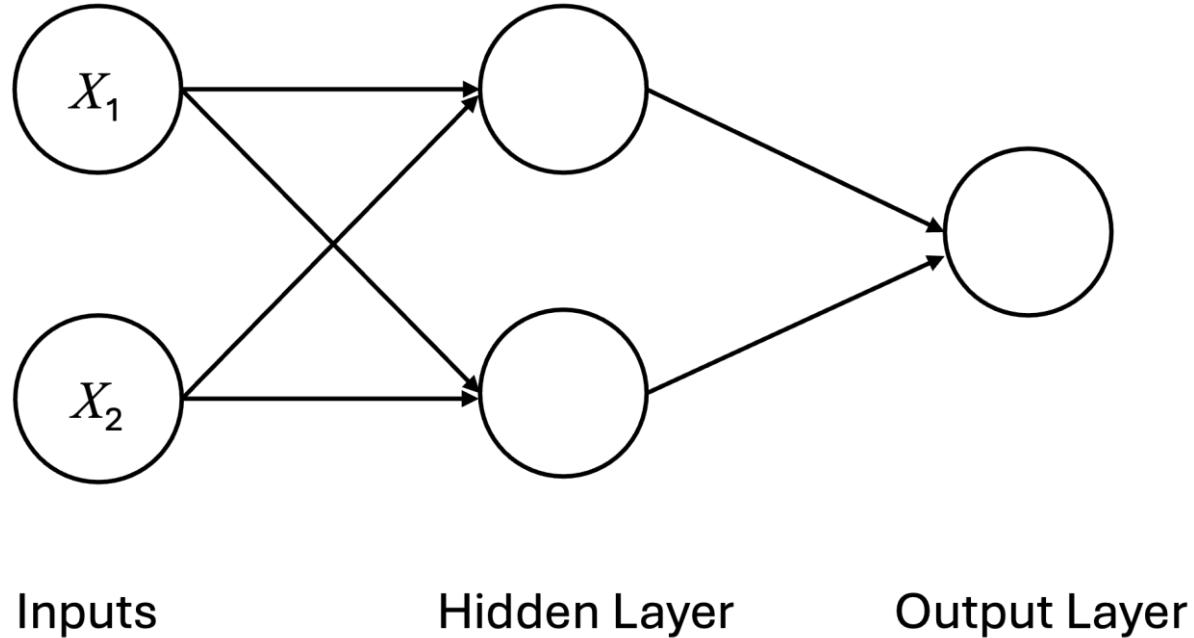


x_1	x_2	y (Class)
0	0	0
1	0	1
0	1	1
1	1	0



Modelling non-linearity

multiple neurons is not enough



Modelling non-linearity

multiple neurons is not enough

Take two linear neurons:

$$f(x) = w_1x + b_1$$

$$g(x) = w_2x + b_2$$


weight bias

Combine them:

$$f(g(x))$$

$$= f(w_2x + b_2)$$

$$= w_1(w_2x + b_2) + b_1$$

$$= (w_1w_2)x + (w_1b_2 + b_1)$$

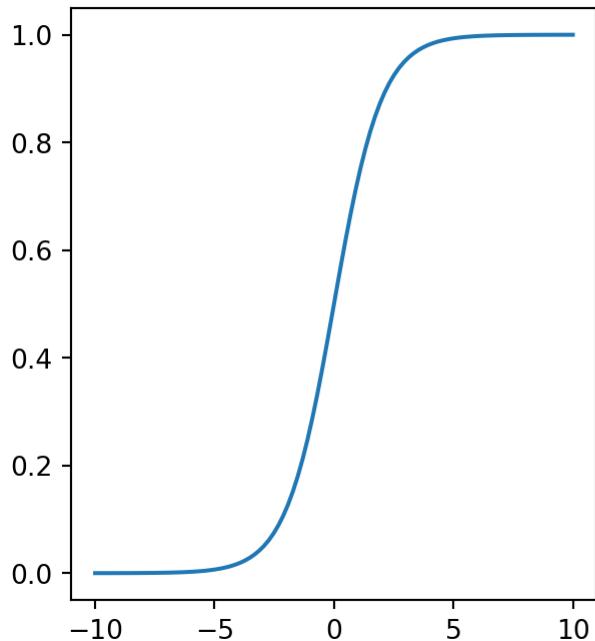
$$= \left(\begin{matrix} w \end{matrix} \right)x + \left(\begin{matrix} b \end{matrix} \right)$$

weight bias

Modelling non-linearity

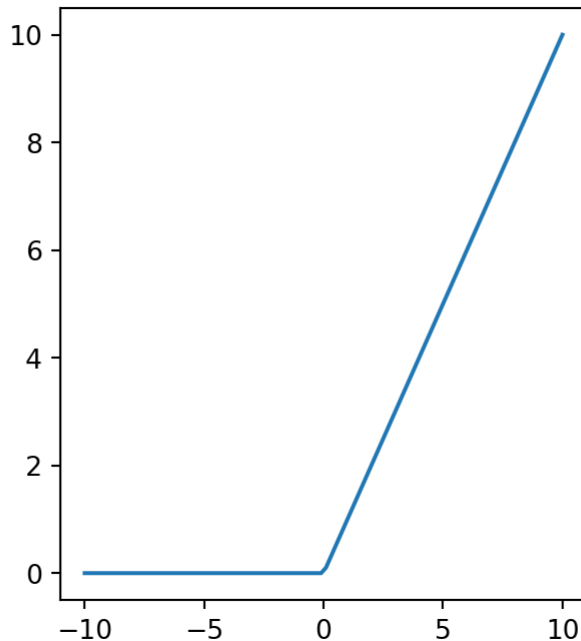
activation functions

Sigmoid



$$\sigma(x) := \frac{1}{1+e^{-x}}$$

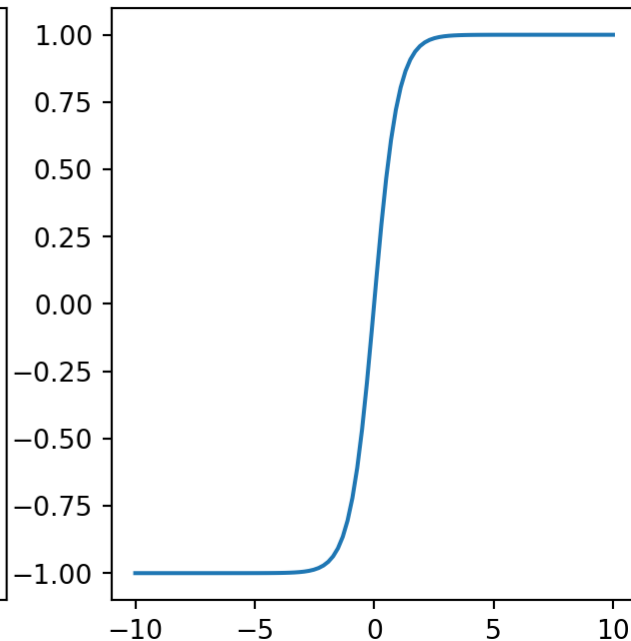
ReLU



$$\text{ReLU}(x) := \max(0, x)$$

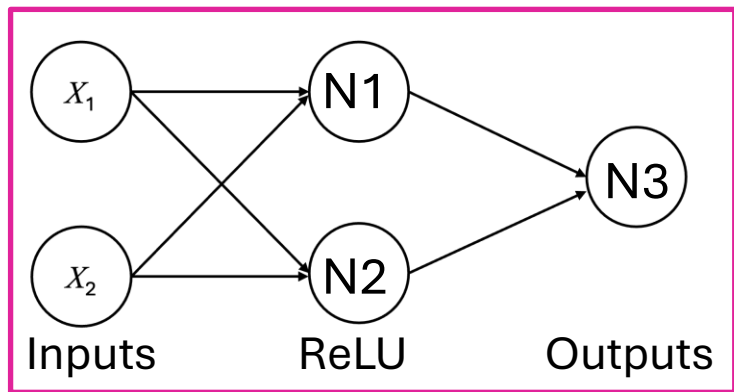
$$= \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

Tanh



$$\tanh(x) := \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Modelling non-linearity

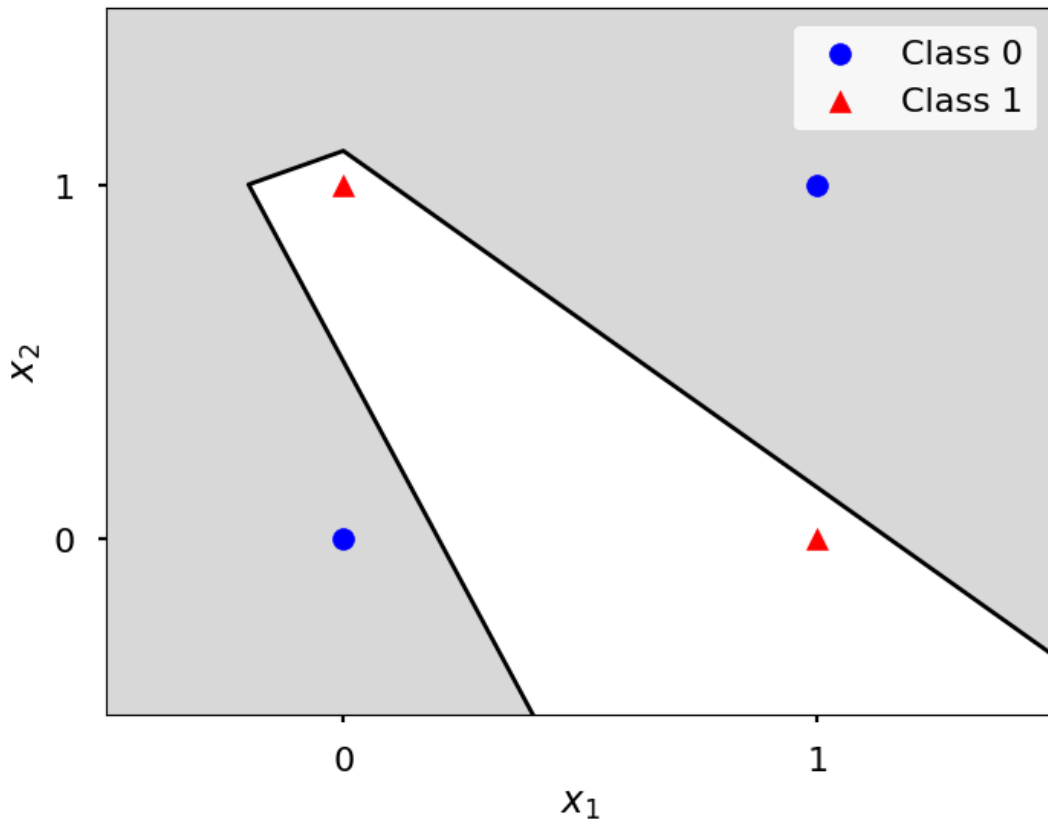


Activation function

Weights and biases

	w1	w2	b
neuron 1	2.0	0.8	0.8
neuron 2	1.2	1.0	-1.0
neuron 3	0.5	-2.5	-0.1

activation functions



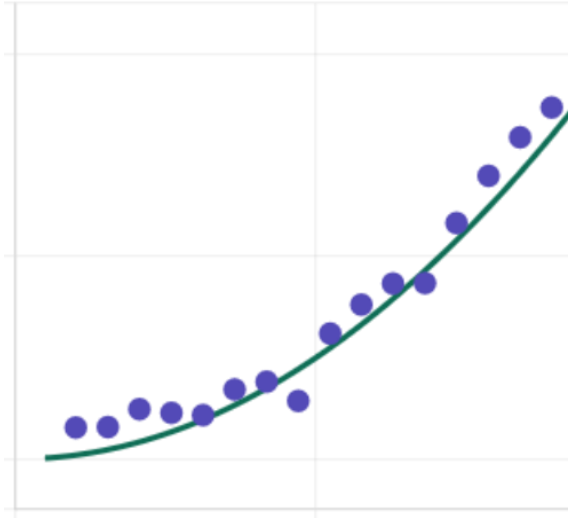
Evaluating a neural network loss (or cost) functions

Model	Loss Function
Regression	Mean Squared Error (MSE)
Classification	Cross-entropy

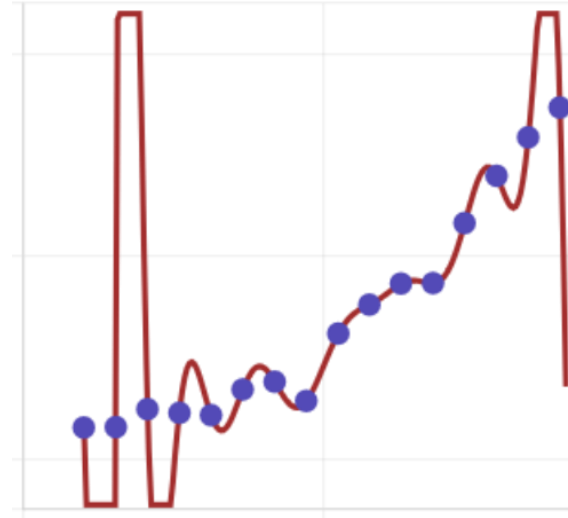
Mean squared error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\text{observation}_i - \text{prediction}_i)^2$$

Out of sample testing



the problem of overfitting



Iris dataset

Sepals

Petals



Iris Versicolor



Iris Setosa



Iris Virginica

Iris dataset

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa

```
from sklearn.datasets import load_iris
```


3 ways of building neural networks

1. From scratch

The hard way

2. Use a framework (PyTorch, Tensorflow, Keras)

Handle the complex maths

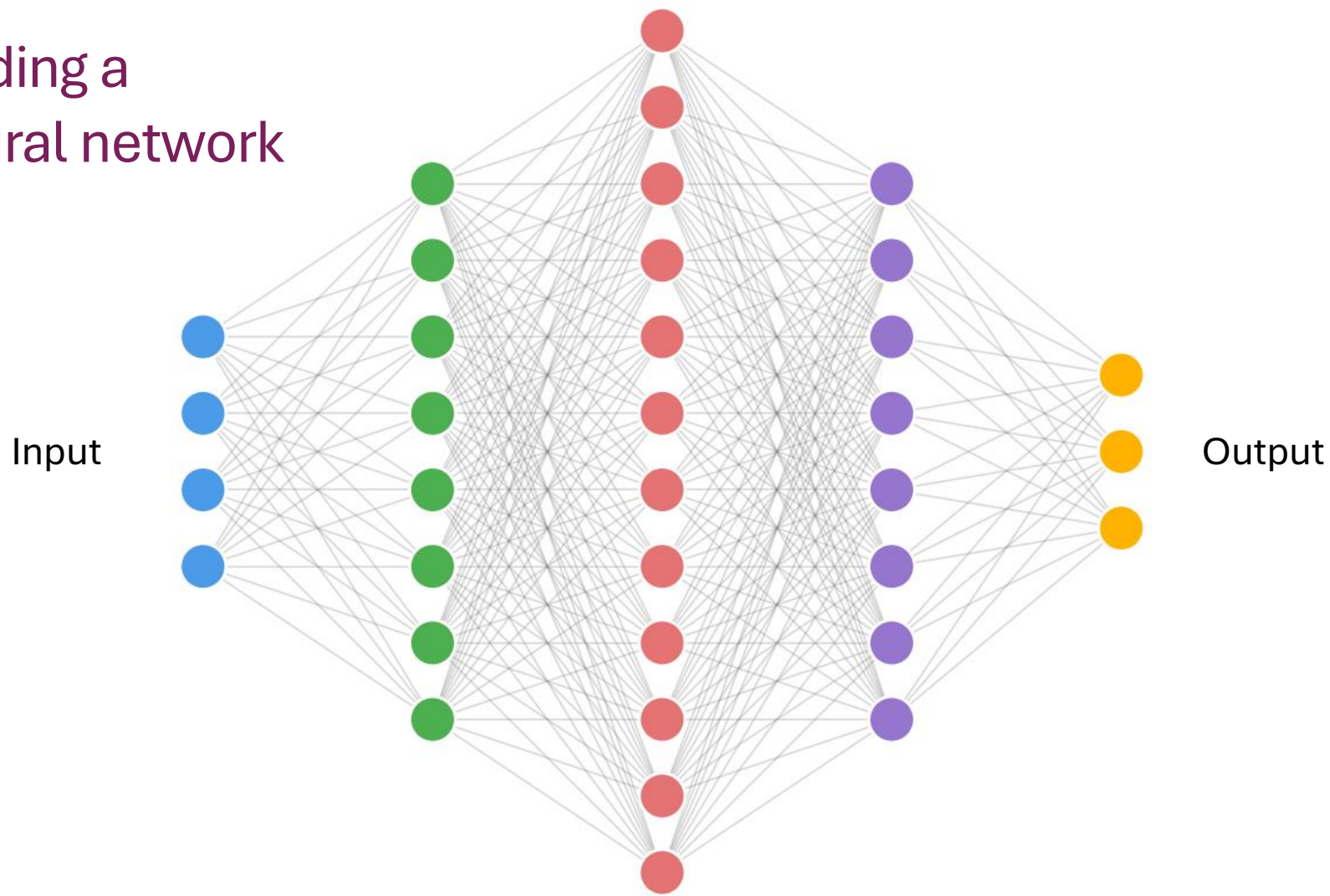
Give you control over the architecture

3. Use a pre-build model

Use it directly or fine-tune

Needs less data

Coding a neural network



Training a neural network

Input data $X_i \longrightarrow$ Neural network(W, b) $\longrightarrow$ Prediction

Recall the loss function

For example, mean squared error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\text{observation}_i - \text{prediction}_i)^2$$

Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Loss (or error)

Training a neural network

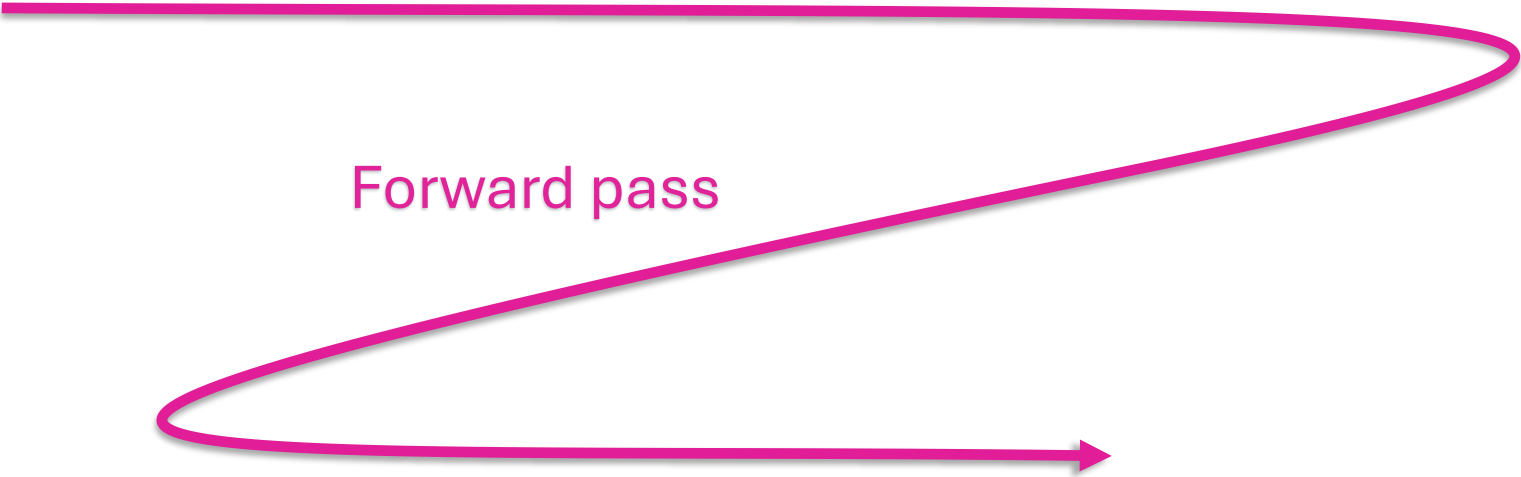
Randomly initialise
weights and biases

Input data $X_i \longrightarrow$ Neural network (W, b) $\longrightarrow$ Prediction

Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Loss (or error)

Training a neural network

Input data $X_i \longrightarrow$ Neural network(W, b) $\longrightarrow$ Prediction

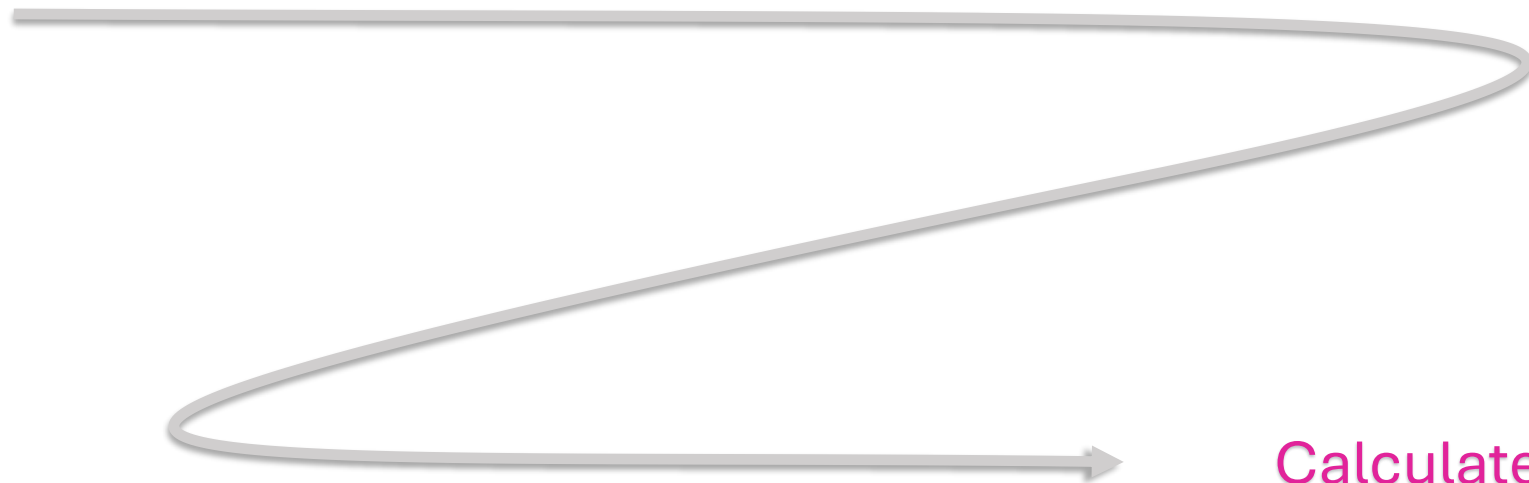


Forward pass

Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Loss (or error)

Training a neural network

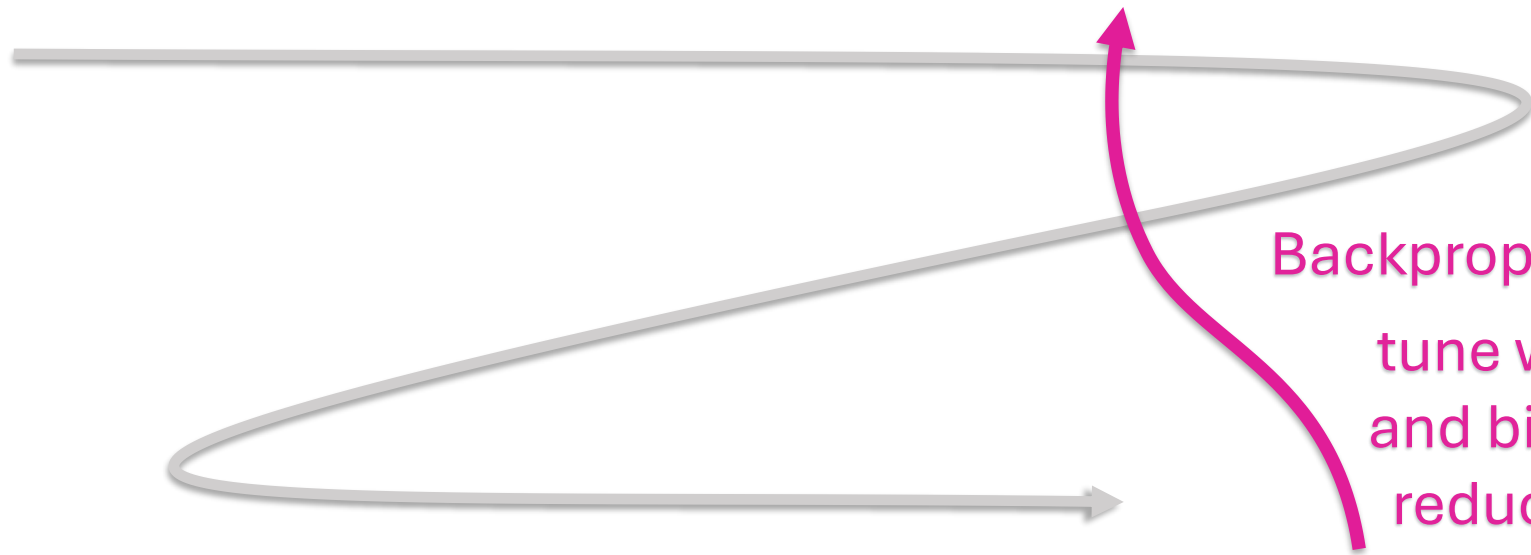
Input data $X_i \longrightarrow$ Neural network(W, b) $\longrightarrow$ Prediction



Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Calculate the loss
Loss (or error)

Training a neural network

Input data $X_i \longrightarrow$ Neural network(W, b) $\longrightarrow$ Prediction



Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Loss (or error)

Training a neural network

Input data $X_i \longrightarrow$ Neural network(W, b) $\longrightarrow$ Prediction

Repeat...

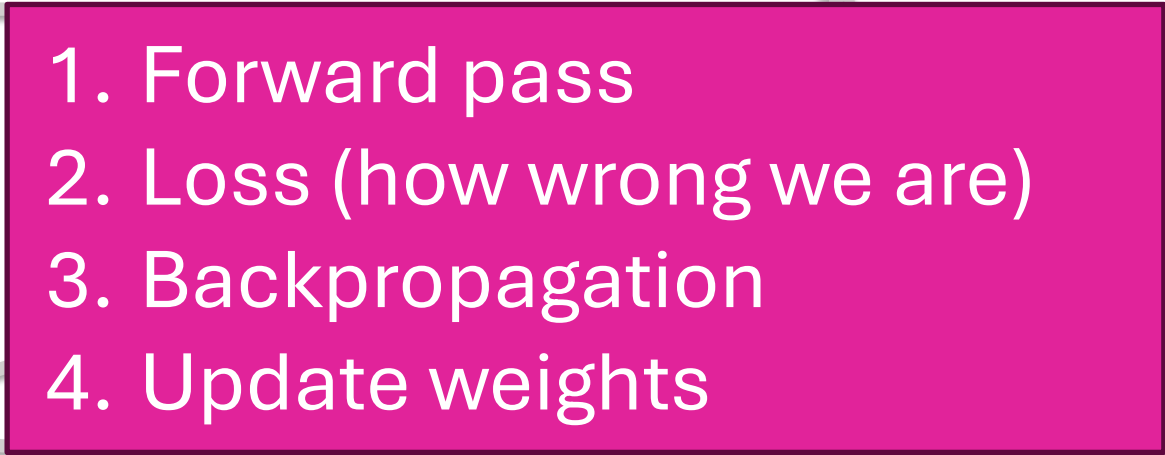
...until loss is low enough

Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Loss (or error)



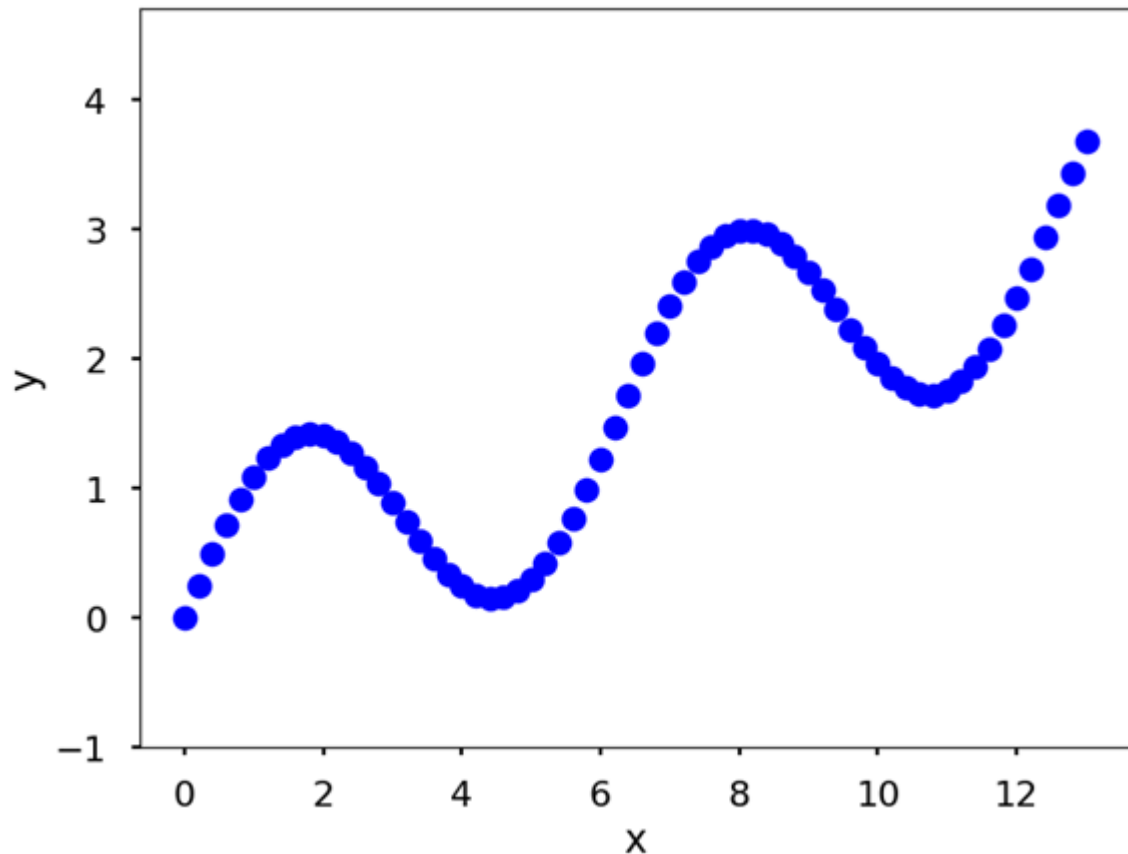
Training a neural network

Input data $X_i \longrightarrow$ Neural network(W, b) $\longrightarrow$ Prediction

- 
1. Forward pass
 2. Loss (how wrong we are)
 3. Backpropagation
 4. Update weights

Predictions, Output $Y_i \longrightarrow$ Loss function $\longrightarrow$ Loss (or error)

Which way to tune the weights?



$$y = w x + b$$

? ?

Which way to tune the weights?

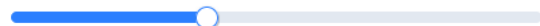
Interactive demo

Tuning a linear fit

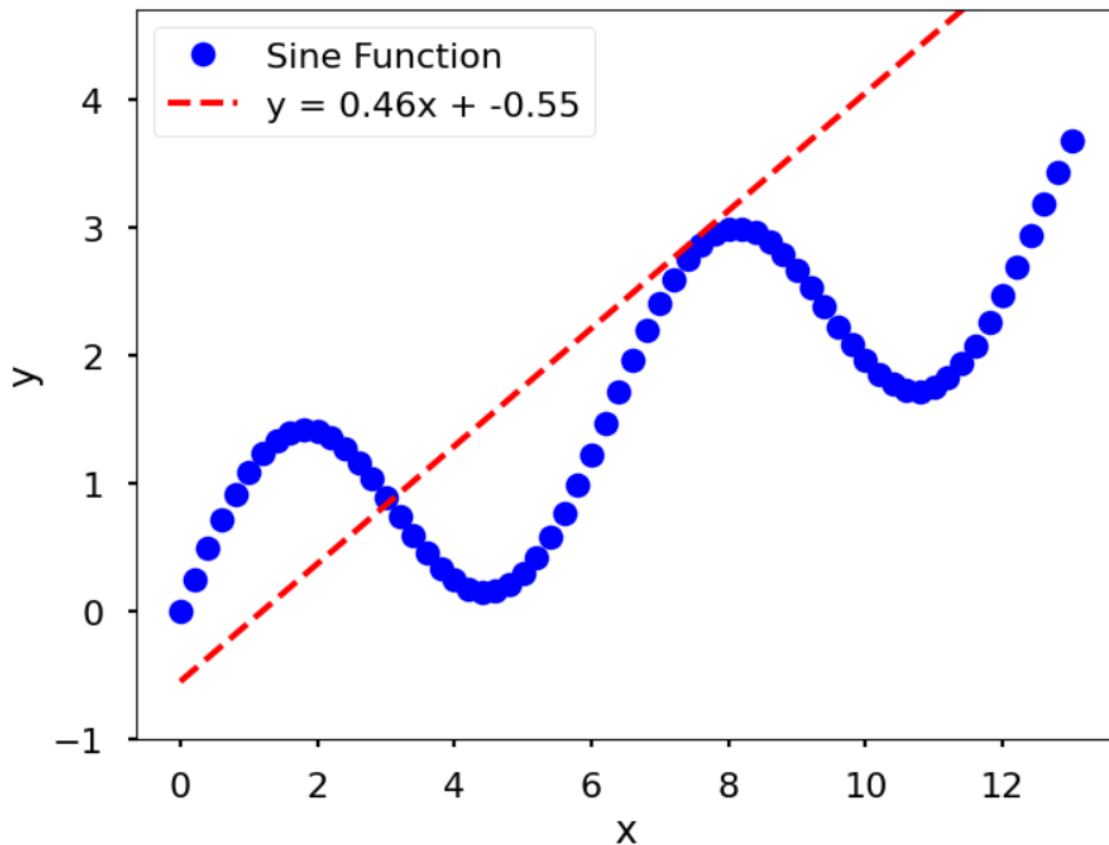
weight = 0.46



bias = -0.55

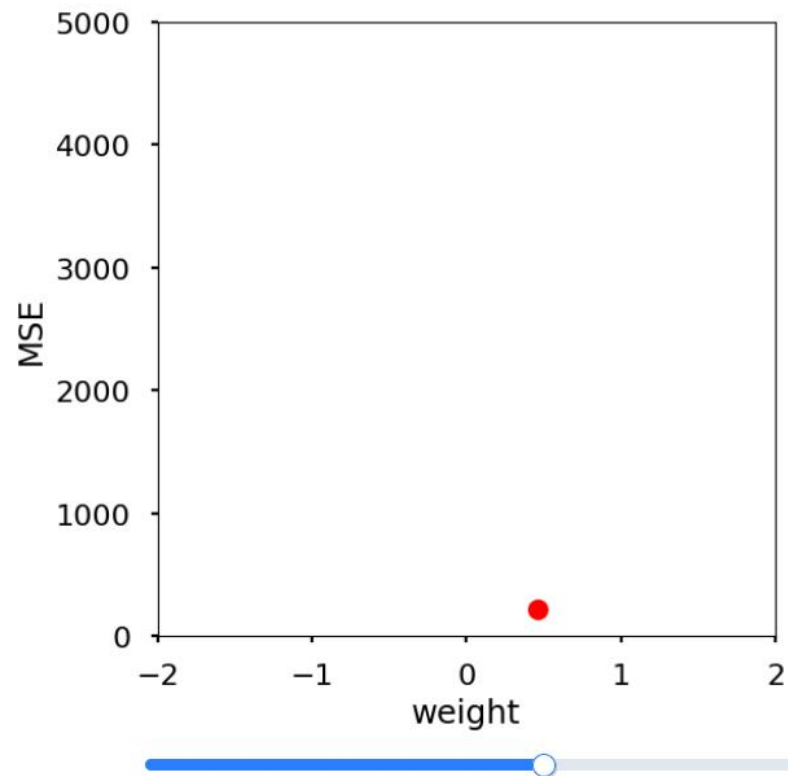
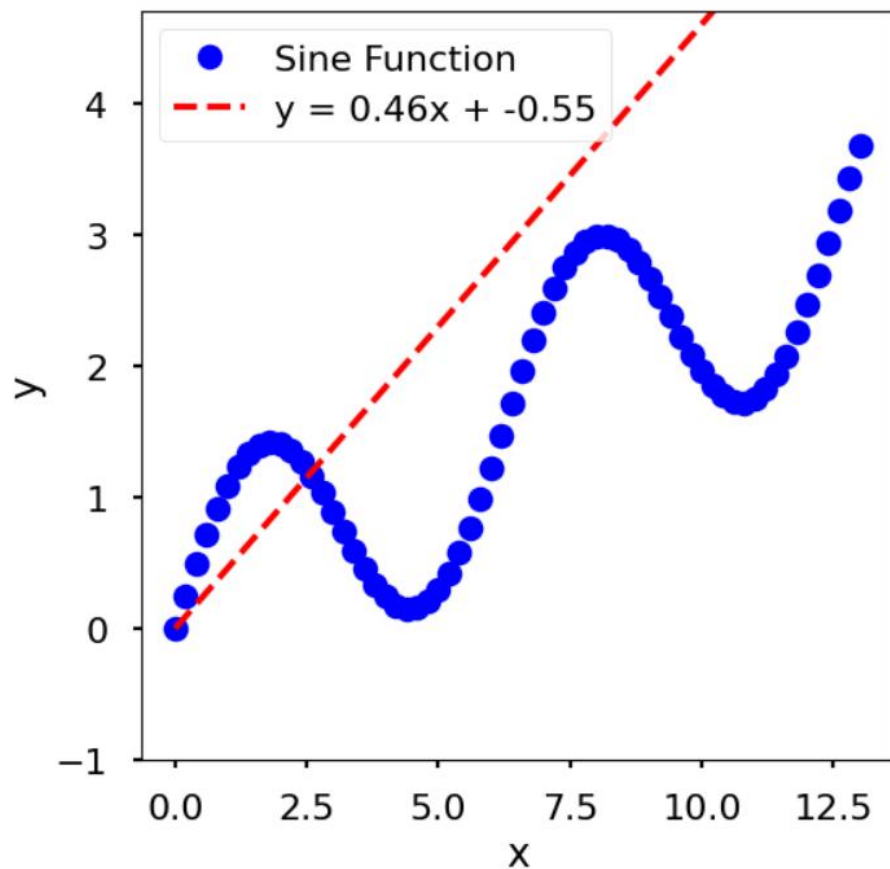


mse = 141.14

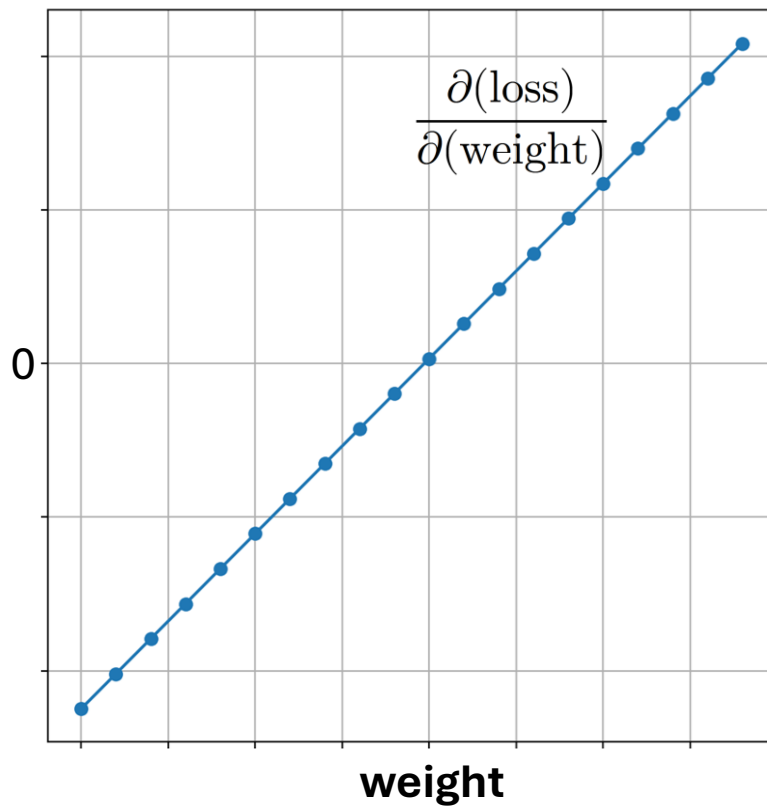
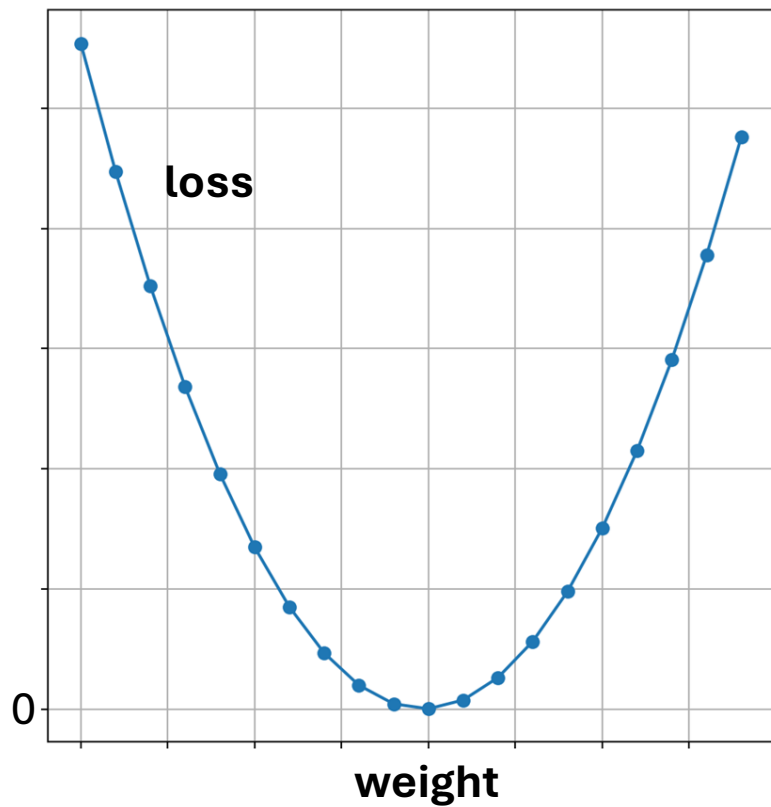


Which way to tune the weights?

Interactive demo



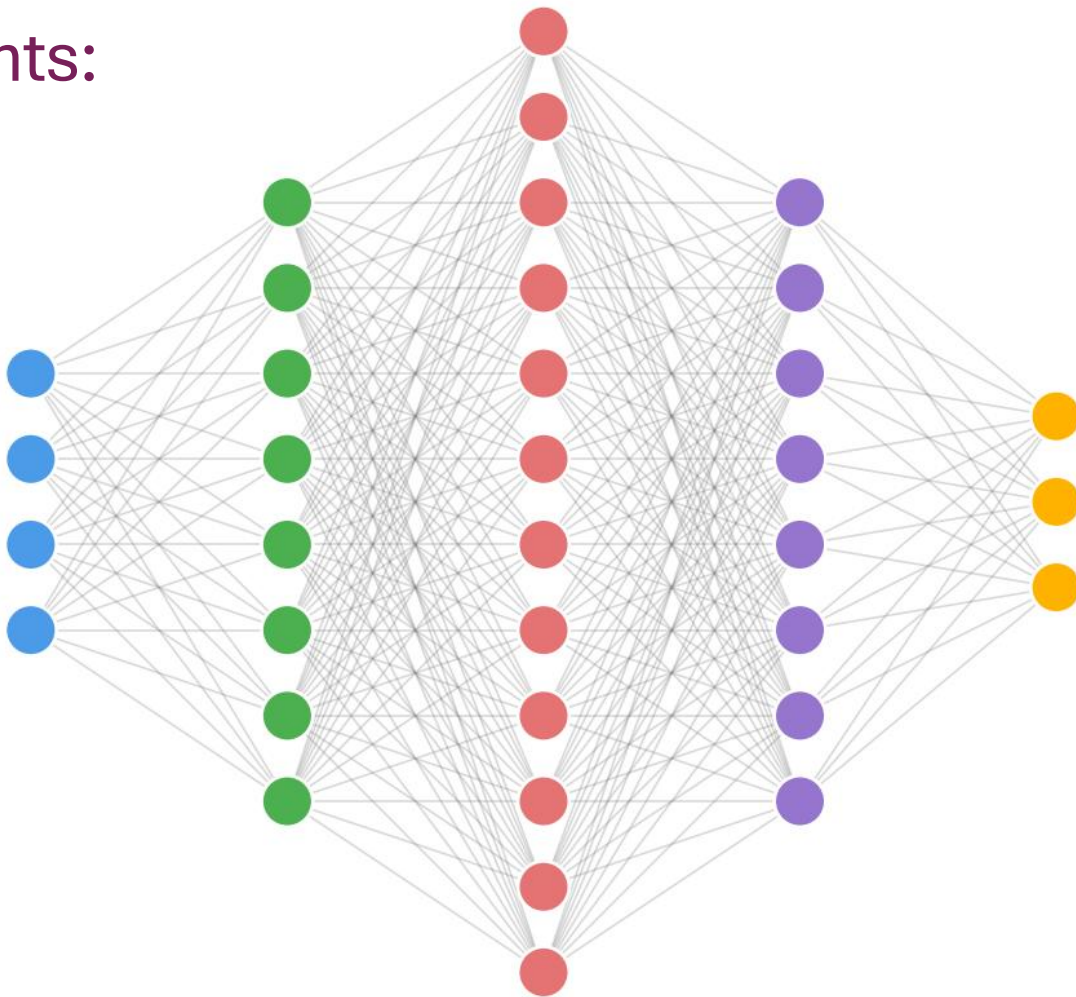
Gradient descent



Calculating the gradients: backpropagation

$$\frac{\partial(\text{loss})}{\partial(\text{weight})}$$

How changing a **weight**
by a small amount
affects the **loss**



Training a real network

Open the XOR example



Epoch
000,222

Activation

Tanh

DATA

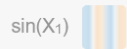
Which dataset do you want to use?



REGENERATE

FEATURES

Which properties do you want to feed in?



+ - 2 HIDDEN LAYERS



2 neurons



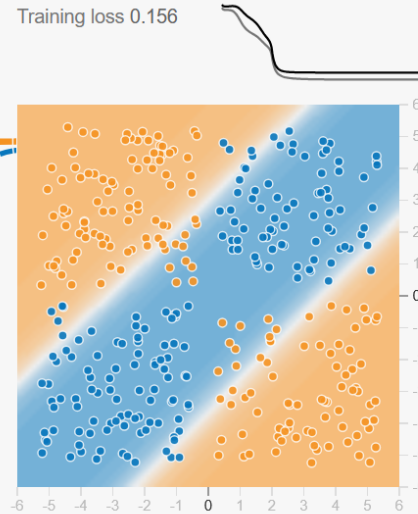
2 neurons

This is the output from one neuron. Hover to see it larger.

The outputs are mixed with varying weights, shown by the thickness of the lines.

OUTPUT

Training loss 0.156



Colors shows data, neuron and weight values.

